

Sommaire :

Introduction

Etape 1

- Ajout base de données
- Structuration du projet
- Création du dépôt GitHub
- Création Kanban sur GitHub

Etape 2

- Création partie Vue de l'application
- Ajout base de données dans l'application
- Création des différents packages

Etape 3

- Codage de la Connexion
- Création de la première documentation technique (chm)
- Codage Partie Personnel
- Codage Partie Absence

Etape 4

- Essai complet du logiciel
- Correction bug

Etape 5

- Ajout d'une fonctionnalité non demandé

Etape 6

- Création de l'Installateur
- Documentation Technique final

Conclusion

Introduction



L'application MediaTek86 est une application qui sera installée sur un poste du service administratif pour pouvoir gérer le personnel d'une médiathèque.

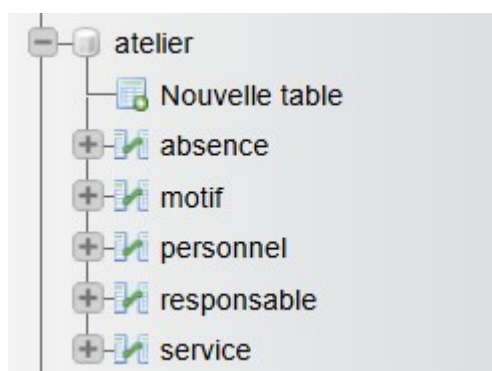
Elle permettra d'afficher la liste du personnel, en ajouter, en supprimer, en modifier, et affichera aussi la liste d'absence d'un personnel sélectionné, et ainsi en ajouter, en supprimer et en modifier.

L'application est écrite en C# sous Visual Studio 2022 Entreprise et exploite une BDD MySQL.

Pour l'instant, elle tournera en local sur la même machine que le SGBD.

Etape 1

Je commence par créer la base de données avec la structure mcd qui nous a été fournie, et je rajoute la table responsable qui a été demandée pour pouvoir se connecter au programme plus tard :



Je remplis ensuite chaque table avec les informations fournies par le CNED, et pour les tables personnel et absence, j'utilise generatedata.com pour pouvoir générer des données aléatoires en grosse quantité.

Génération des données de la table personnel :

5	Data Type	Column Name	Examples	Options
#1	Names	nom	Alex (any gender)	Name X
#2	Names	prenom	Smith (surname)	Surname X
#3	Phone / Fax	tel	France	06XXXXXXXX X
#4	Email	mail	No examples available.	SOURCE: RANDOM
#5	Number Range	idservice	No examples available.	Between 1 and 3

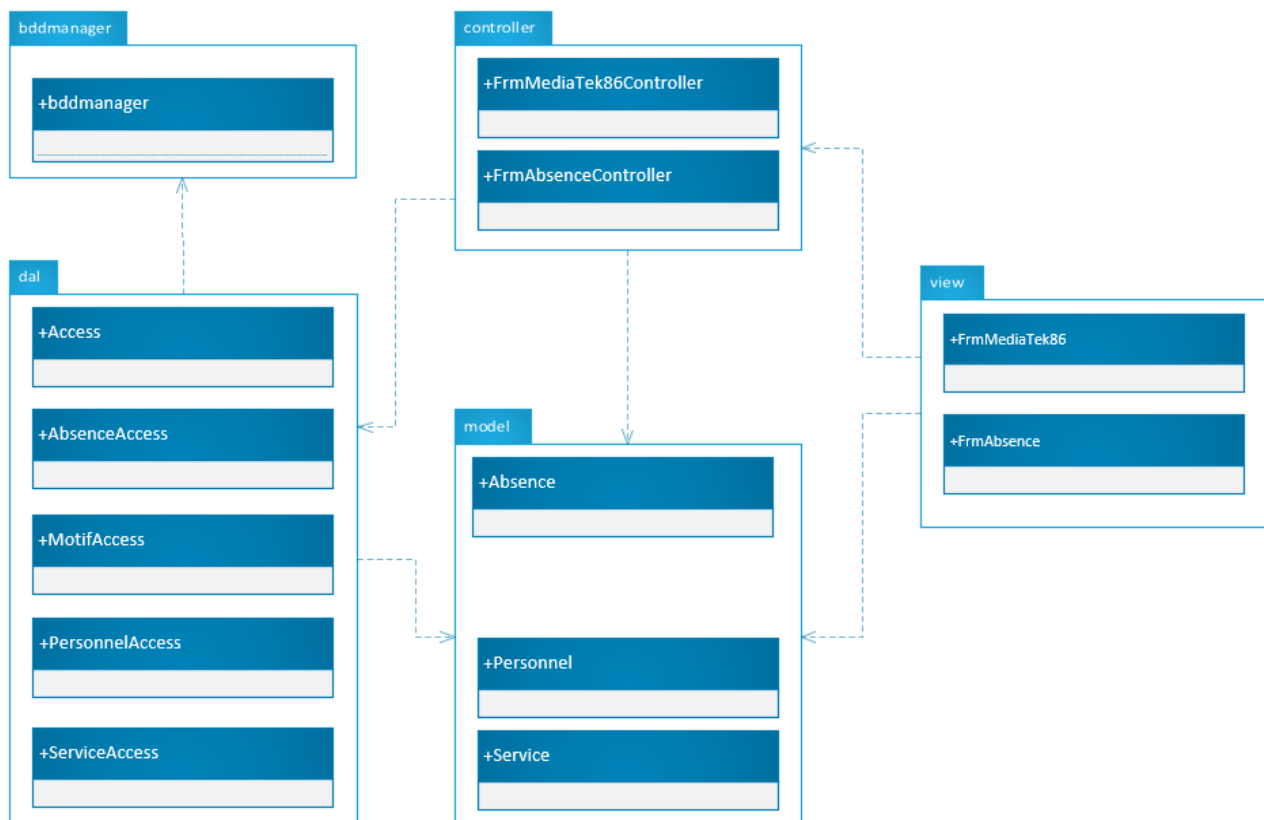
Add

1

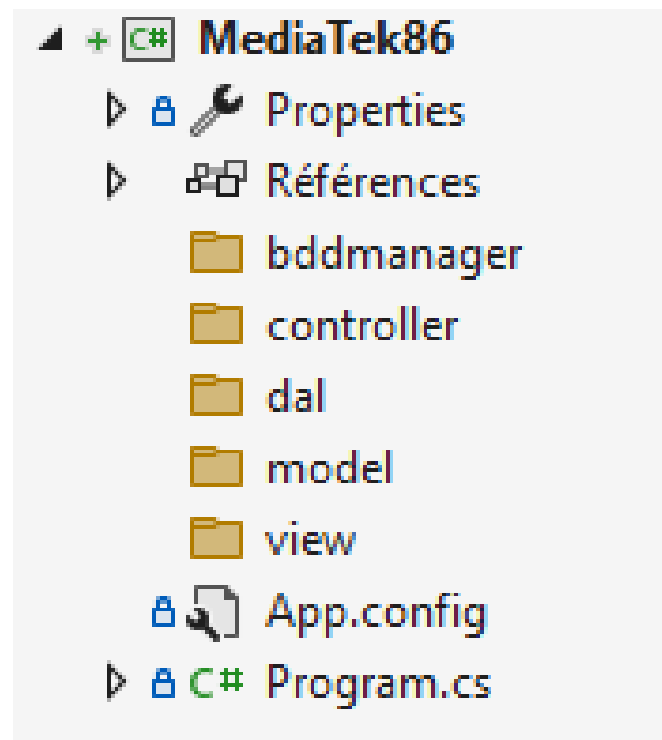
ROW

Etc..

Je passe ensuite à la création du diagramme de paquetage pour pouvoir avoir une vue global sur ce que je vais devoir créer :



Une fois le diagramme de paquetage structuré, je passe à la création de mes fichiers dans mon programme.

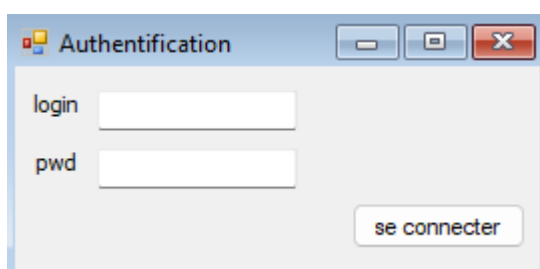


Je crée ensuite mon répertoire GitHub dans lequel je pusherais l'intégralité du programme lorsque je le commencerais et que j'y rajouterais des fonctionnalités petit-à-petit pour toujours garder une trace de mon avancée.

Ensuite, je crée mon Kanban sur mon répertoire github pour pouvoir garder une trace de mon avancée en temps réel afin de savoir que faire par la suite.

Etape 2

Je passe ensuite à la création de la vue de l'application, c'est-à-dire les trois interfaces graphiques clés du programme (l'interface pour s'authentifier, l'interface pour gérer le personnel et l'interface pour gérer les absences d'un personnel.)



Personnel

liste du personnel

modifier supprimer absences

ajouter un personnel

nom mail

prenom tel

service

enregistrer annuler

Absence

absence d'un personnel

nom : Garcia prénom : Antoine modifier supprimer

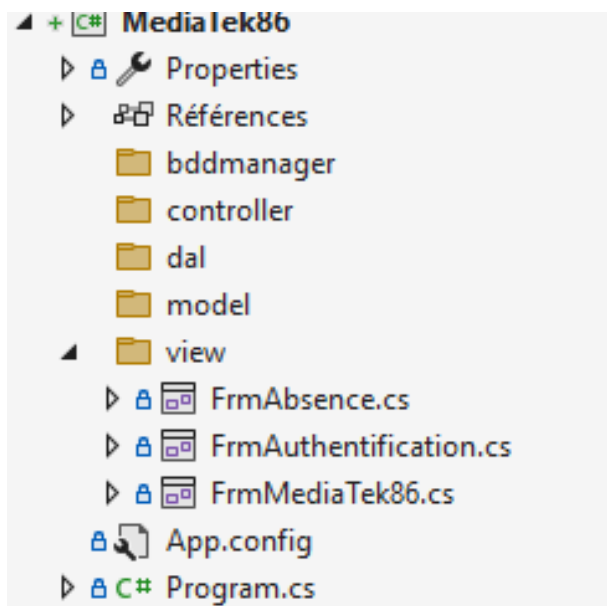
ajouter une absence

date de début : jeudi 28 mai 2026 motif :

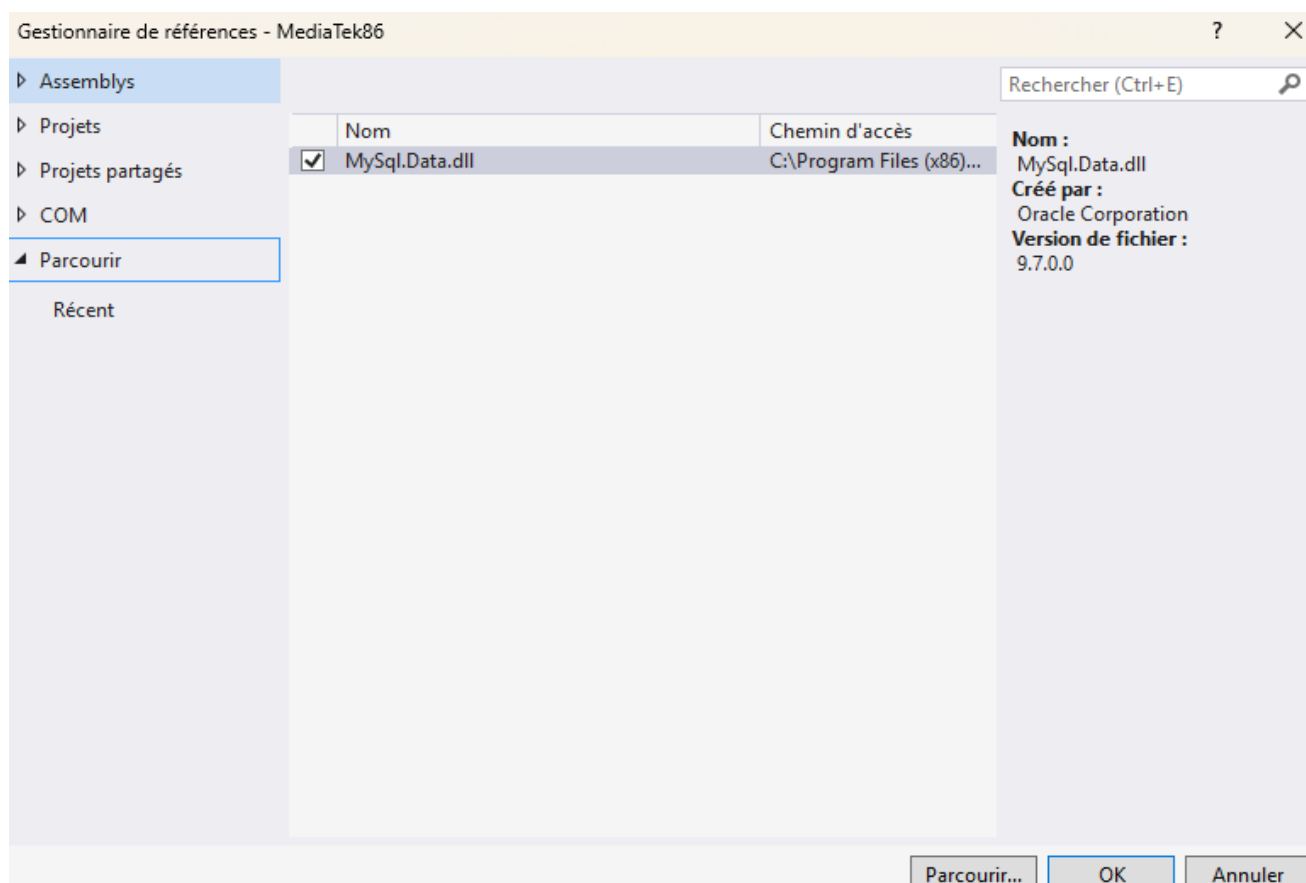
date de fin : jeudi 28 mai 2026

enregistrer annuler

Voici la vue de mon projet actuellement :



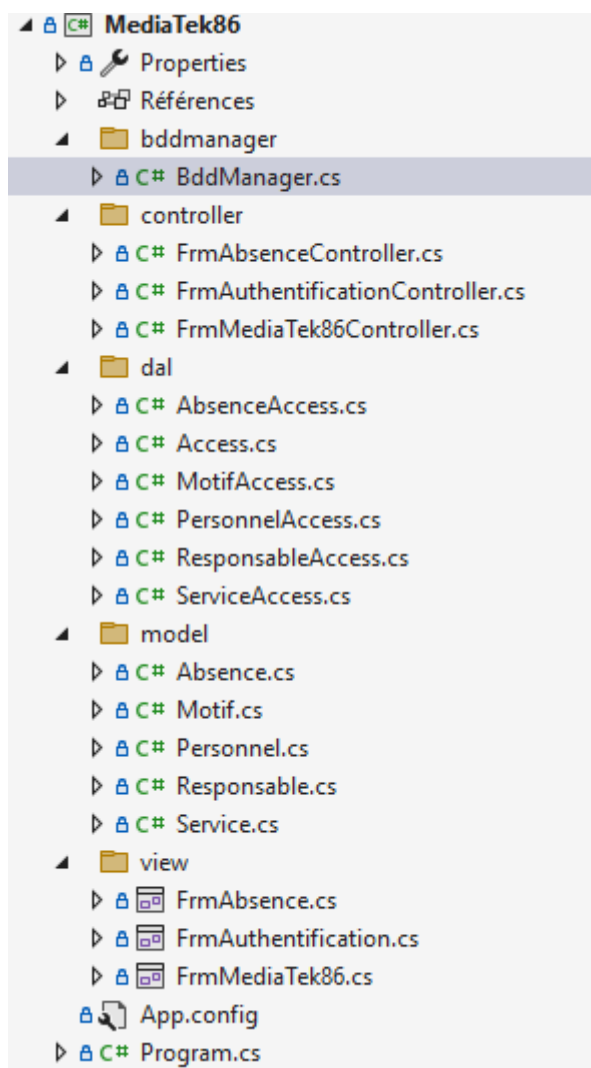
J'ajoute ensuite le fichier bddmanager dans le dossier bddmanager afin de pouvoir se connecter à la base de données et effectuer des requêtes SQL tout en pensant en ajoutant la référence MySQL pour que le fichier bddmanager fonctionne correctement. J'ajoute aussi le fichier Access.cs dans dal pour pouvoir avoir le serveur, l'id, le mot de passe et la base de données utilisé pour pouvoir s'y connecter.



Je fais mes premier commits pour pouvoir garder une trace et une sauvegarde de tout ça.

Je passe ensuite à la création de l'intégralité des classes dont je vais avoir besoin pour la suite du développement du programme. Je commit ensuite sur GitHub.

Suite à la création des classes, voici à quoi ressemble mon projet :



Etape 3

Une fois l'intégralité de mes classes créées et de mes interfaces correctement faite, je passe au codage de l'application.

Je code donc la connexion au programme en vérifiant que le programme vérifie bien que la personne qui se connecte possède un compte responsable dans la base de données.

```
/// <summary>
/// Permet de se connecter à l'application en vérifiant les identifiants du responsable
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
1 référence
private void btnConnect_Click(object sender, EventArgs e)
{
    String login = txtLogin.Text;
    String pwd = txtPwd.Text;
    if (String.IsNullOrEmpty(login) || String.IsNullOrEmpty(pwd))
    {
        MessageBox.Show("Tous les champs doivent être remplis.", "Erreur");
    }

    else
    {
        Responsable resp = new Responsable(login, pwd);
        if (controller.ControleAuthentification(resp))
        {
            FrmMediaTek86 frm = new FrmMediaTek86();
            frm.ShowDialog();
        }
        else
        {
            MessageBox.Show("Identifiants incorrects.", "Erreur");
        }
    }
}
```

```
/// <summary>
/// Permet de savoir si l'utilisateur peut se connecter ou non (login et pwd)
/// </summary>
/// <param name="login"></param>
/// <param name="pwd"></param>
/// <returns>vrai si les identifiants sont bons</returns>
public Boolean ControleAuthentification(Responsable responsable)
{
    if (access.Manager != null)
    {
        string req = "select * from responsable ";
        req += "where login=@login and pwd=SHA2(@pwd, 256)";
        Dictionary<string, object> parameters = new Dictionary<string, object>();
        parameters.Add("@login", responsable.Login);
        parameters.Add("@pwd", responsable.Pwd);
        try
        {
            List<Object[]> records = access.Manager.ReqSelect(req, parameters);
            if (records != null)
            {
                return (records.Count > 0);
            }
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
            Environment.Exit(0);
        }
    }
    return false;
}
```

```

internal class FrmAuthentificationController
{
    /// <summary>
    /// récupère les opérations possibles sur Responsable
    /// </summary>
    private readonly ResponsableAccess responsableAccess;

    /// <summary>
    /// Récupère l'accès aux données
    /// </summary>

    public FrmAuthentificationController()
    {
        responsableAccess = new ResponsableAccess();
    }

    /// <summary>
    /// Permet de contrôler l'authentification
    /// </summary>
    /// <param name="responsable"></param>
    /// <returns></returns>

    public Boolean ControleAuthentification(Responsable responsable)
    {
        return responsableAccess.ControleAuthentification(responsable);
    }
}

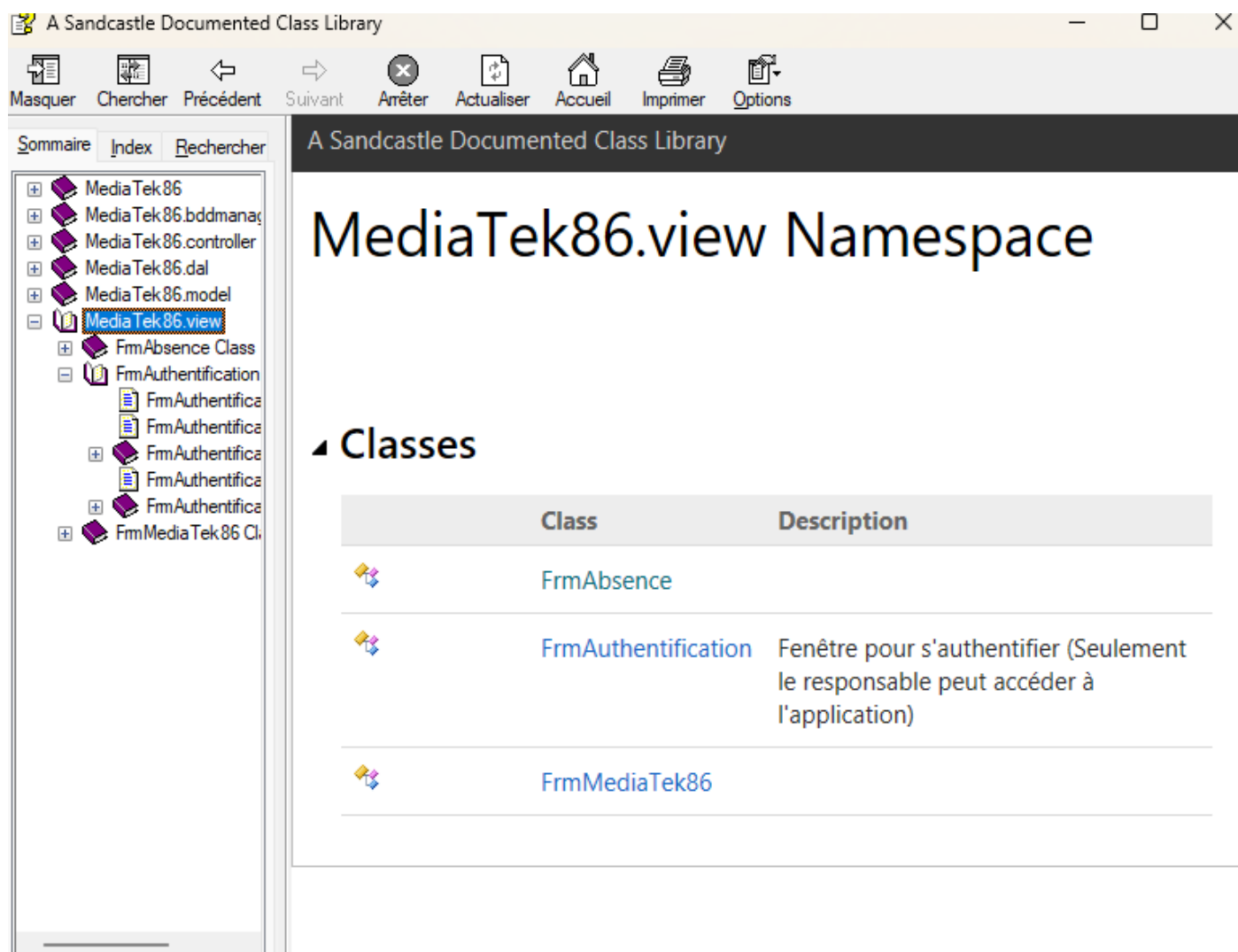
namespace MediaTek86.model
{
    /// <summary>
    /// Classe métier interne pour pouvoir mémoriser les informations de connexions
    /// </summary>
    5 références
    internal class Responsable
    {
        2 références
        public string Login { get; }
        2 références
        public string Pwd { get; }

        /// <summary>
        /// Mettre valeur dans propriétés de la classe responsable
        /// </summary>
        /// <param name="login"></param>
        /// <param name="pwd"></param>
        1 référence
        public Responsable(string login, string pwd)
        {
            this.Login = login;
            this.Pwd = pwd;
        }
    }
}

```

btnConnect_click permet d'envoyer une requête (ControleAuthentification) au controller qui va appeler la dal pour pouvoir vérifier si les identifiants entrés correspondent à un responsable dans la table responsable de la base de données. Si les identifiants sont corrects, alors on ouvre l'interface graphique FrmMediaTek86 qui nous servira à gérer la liste du personnel.

Je commit sur GitHub mon avancé et je met à jour le Kanban puis je crée ma première documentation technique du programme à l'aide de SandCastle.



Je passe ensuite au codage de la partie Personnel.

Je commence donc par afficher la liste du Personnel présent en base de données puis par ajouter la liste des services dans la ComboBOX « service » pour préparer l'ajout d'un personnel.

liste du personnel

	Nom	Prenom	Tel	Mail	Service
	Ahmed	Fulton	0667627188	orci.tincidunt@aol.net	médiation culturelle
▶	Briar	Hopper	0698872877	integer.sem.elit@outlook.com	médiation culturelle
	Caesar	Glover	0642588182	aliquam.vulputate@aol.com	médiation culturelle
	Cally	Floyd	0682448567	consectetur.euismod.est@hotmail.edu	administratif
	Evan	Keller	0662723748	curabitur.ut@protonmail.couk	prêt
	Josiah	Avery	0644212881	nisi.aenean.eget@protonmail.edu	médiation culturelle

modifier supprimer absences

ajouter un personnel

nom mail

premier tel

service

administratif
administratif
médiation culturelle
prêt

enregistrer annuler

Ensuite, j'ajoute la fonctionnalité pour pouvoir ajouter un personnel.

J'ajoute donc la méthode btnEnregPers_Click dans la classe FrmMediaTek86 pour pouvoir effectuer une requête lors de la pression du bouton enregistrer.

J'ajoute la méthode AddPersonnel dans le controller ainsi que dans le dal pour que les deux puissent communiquer et ainsi permettre l'ajout d'un personnel dans la base de données si tout est valide.

Le code étant fonctionnel, je passe directement à l'ajout de la modification d'un personnel. Je crée donc la méthode UpdatePersonnel dans le controller et la dal, puis dans la classe FrmMediaTek86 je modifie mon code pour pouvoir l'intégrer au sein de l'ajout d'un personnel avec un booléen EnCoursDeModifPersonnel pour pouvoir vérifier si le programme est en mode modification d'un personnel ou ajout d'un personnel. J'ajoute aussi une confirmation de modification. J'ajoute une méthode btnDemandeModifPers_click pour pouvoir directement remplir les informations du personnel automatiquement dans la GroupBox Personnel. J'ai aussi ajouté une méthode EnCoursModifPersonnel en booléen true/false pour pouvoir changer le nom de la groupBox pour pouvoir afficher « modifier un personnel » plutôt que « ajouter un personnel ».

```
/// <summary>
/// Permet de remplir les champs de la partie modification avec les informations du personnel sél
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
1 référence
private void btnDemandeModifPers_Click(object sender, EventArgs e)
{
    if (dgvPersonnel.SelectedRows.Count > 0)
    {
        Personnel personnel = (Personnel)bdgPersonnel.List[bdgPersonnel.Position];
        EnCoursModifPersonnel(true);
        txtNom.Text = personnel.Nom;
        txtPrenom.Text = personnel.Prenom;
        txtTel.Text = personnel.Tel;
        txtMail.Text = personnel.Mail;
        cboService.SelectedIndex = cboService.FindStringExact(personnel.Service.Nom);
    }
    else
    {
        MessageBox.Show("Une ligne doit être sélectionnée.", "Information");
    }
}
```



```

/// <summary>
/// Affichage de la modification d'un personnel si jamais on le modifie, sinon affichage ajout d'un personnel
/// </summary>
/// <param name="modif"></param>
4 références
private void EnCoursModifPersonnel(Boolean modif)
{
    enCoursDeModifPersonnel = modif;
    grbLePersonnel.Enabled = !modif;
    if (modif)
    {
        grbPersonnel.Text = "modifier un personnel";
    }
    else
    {
        grbPersonnel.Text = "ajouter un personnel";
        txtNom.Text = "";
        txtPrenom.Text = "";
        txtTel.Text = "";
        txtMail.Text = "";
    }
}

/// <summary>
/// Permet d'enregistrer un nouveau personnel
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
1 référence
private void btnEnregPers_Click(object sender, EventArgs e)
{
    if (!txtNom.Text.Equals("") && !txtPrenom.Text.Equals("") && !txtTel.Text.Equals("") && !txtMail.Text.Equals("") && cboService.SelectedIndex != -1)
    {
        Service service = (Service)bdgServices.List[bdgServices.Position];
        Personnel personnel = (Personnel)bdgPersonnel.List[bdgPersonnel.Position];
        if (MessageBox.Show("Voulez-vous vraiment modifier les informations de " + personnel.Nom + " " + personnel.Prenom + " ?", "Confirmation de modification", MessageBoxButtons.YesNo) == DialogResult.Yes)
        {
            if (enCoursDeModifPersonnel)
            {
                personnel.Nom = txtNom.Text;
                personnel.Prenom = txtPrenom.Text;
                personnel.Tel = txtTel.Text;
                personnel.Mail = txtMail.Text;
                personnel.Service = service;
                controller.UpdatePersonnel(personnel);
            }
            else
            {
                personnel = new Personnel(0, txtNom.Text, txtPrenom.Text, txtTel.Text, txtMail.Text, service);
                controller.AddPersonnel(personnel);
            }
            RemplirListePersonnel();
        }
        EnCoursModifPersonnel(false);
    }
    else
    {
        MessageBox.Show("Tous les champs doivent être remplis.", "Information");
    }
}

```

J'ajoute ensuite la méthode btnAnnulPers_click dans la classe FrmMediaTek86 pour pouvoir annuler une modification en cours ou un enregistrement en cours.

Pour se faire, elle appelle simplement la méthode EnCoursModifPersonnel en false puisque celle-ci remet à zéro l'intégralité de la GroupBox lorsqu'un personnel a fini sa modification.

Pour en finir avec la section personnel, j'ajoute le fait de pouvoir supprimer un personnel sélectionné. Je code donc la méthode `btnDemandeSuppPers_click` dans `FrmMediaTek86`, qui demande une confirmation de suppression, et qui enverra une requête au controller qui appellera la dal qui se prénomme `DelPersonnel` et qui supprime donc le personnel sélectionné de la base de données.

Une fois la section Personnel correctement fini (ajouter, supprimer, modifier, annuler), je peux désormais passer à la section Absence du programme.

Comme pour la partie Personnel, je dois d'abord m'assurer de récupérer la liste des absences dans la base de données. Mais cette fois je dois rajouter quelque chose, c'est la récupération du personnel sélectionné. Pour ce faire, dans la classe `FrmMediaTek86`, je crée la méthode `btnDemandeAbsPers_Click` en vérifiant qu'un personnel est bien et bien sélectionné dans la liste, et j'ouvre l'interface graphique `FrmAbsence` en lui ajoutant en paramètre (`personnelSelectionne`), qui est donc le personnel sélectionné dans la liste des personnels.

```
1 référence
private void btnDemandeAbsPers_Click(object sender, EventArgs e)
{
    if (dgvPersonnel.SelectedRows.Count > 0)
    {
        Personnel personnelSelectionne = (Personnel)bdgPersonnel.List[bdgPersonnel.Position];
        FrmAbsence frm = new FrmAbsence(personnelSelectionne);
        frm.ShowDialog();
    }

    else
    {
        MessageBox.Show("Veuillez sélectionner une ligne.", "Information");
    }
}
```

Une fois dans la classe `FrmAbsence`, je récupère le personnel sélectionné pour pouvoir l'utiliser.

4 references

```
public partial class FrmAbsence : Form
{
    /// <summary>
    /// Variable pour pouvoir stocker le personnel sélectionner
    /// </summary>
    private Personnel personnelRecu;
```

Je crée ensuite une valeur Int idPersonnel dans le controller FrmAbsenceController puisque je ne peux pas envoyer le personnelRecu tel quel donc il faut que je le stocke dans le controller pour ensuite l'envoyer dans le dal et récupérer les données du personnel sélectionné. Pour se faire, j'ai donc créé la méthode GetAbsence dans le controller et dans le dal, que j'appelle dans la méthode RemplirListeAbsence de la classe FrmAbsence.

```
/// <summary>
/// Afficher la liste des absences du personnel sélectionner
/// </summary>
1 référence
public void RemplirListeAbsence()
{
    controller.idPersonnel = personnelRecu.Idpersonnel;
    List<Absence> Absences = controller.GetAbsence();
    bdgAbsence.DataSource = Absences;
    dgvAbsences.DataSource = bdgAbsence;
    dgvAbsences.Columns["Idpersonnel"].Visible = false;
    dgvAbsences.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.AllCells;
}
```

3 references

```
internal class FrmAbsenceController
{
```

```
/// <summary>
/// récupère les opérations possibles sur les absences
/// </summary>
private readonly AbsenceAccess absenceAccess;
```

```
/// <summary>
/// Création d'une valeur idPersonnel pour stocker l'id du personnel dont on veut récupérer les absences
/// </summary>
```

2 références

```
public int idPersonnel { get; set; }
```

```
/// <summary>
/// Récupère et retourne les absences d'un personnel à partir de son id
/// </summary>
/// <returns></returns>
```

1 référence

```
public List<Absence> GetAbsence()
{
    return absenceAccess.GetAbsence(this.idPersonnel);
}
```

```

/// <summary>
/// Récupère et retourne les absences d'un personnel par son id
/// </summary>
/// <returns>liste des absences d'un personnel par son id</returns>
1 référence
public List<Absence> GetAbsence(int idPersonnel)
{
    List<Absence> Absence = new List<Absence>();
    if (access.Manager != null)
    {
        string req = "SELECT a.idpersonnel, a.datedebut, a.datefin, a.idmotif, ";
        req += "p.nom, p.prenom, p.tel, p.mail, s.idservice, s.nom, m.libelle ";
        req += "FROM absence a ";
        req += "JOIN personnel p ON a.idpersonnel = p.idpersonnel ";
        req += "JOIN service s ON p.idservice = s.idservice ";
        req += "JOIN motif m ON a.idmotif = m.idmotif ";
        req += "WHERE a.idpersonnel = " + idPersonnel + " ";
        req += "ORDER BY a.datedebut DESC, a.datefin DESC;";
        try
        {
            List<Object[]> records = access.Manager.ReqSelect(req);
            if (records != null)
            {
                foreach (Object[] record in records)
                {
                    Service idservice = new Service((int)record[8], (string)record[9]);
                    Personnel idpersonnel = new Personnel((int)record[0], (string)record[4], (string)record[5], (string)record[6], (string)record[7], idservice);
                    Motif idmotif = new Motif((int)record[3], (string)record[10]);
                    Absence absence = new Absence(idpersonnel, (DateTime)record[1], (DateTime)record[2], idmotif);
                    Absence.Add(absence);
                }
            }
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
            Environment.Exit(0);
        }
    }
    return Absence;
}

```

Bien évidemment, j'ai déjà créé le model Absence et le model Motif pour pouvoir correctement les récupérer, et j'ai bien fait attention de lier correctement l'idPersonnel et l'idMotif tout en respectant l'ajout de service pour pouvoir correctement recevoir le personnel reçu car sans le service, je ne pouvais pas correctement recevoir le personnel reçu et je ne pouvais pas le lier aux absences avec son idpersonnel.

Avec ça, j'ai désormais dans mon interface graphique FrmAbsence la liste d'un personnel sélectionné dans la liste personnel de l'interface graphique FrmMediaTek86 (Gestion du personnel)

absence d'un personnel

	date_de_debut	date_de_fin	motif
▶	06/04/2028 21:48	29/01/2029 21:48	congé parental
	02/01/2027 19:59	06/06/2026 06:28	motif familial
	02/11/2026 18:51	04/10/2025 14:53	maladie
	05/05/2026 01:47	12/09/2026 17:48	maladie
	06/04/2026 11:10	31/05/2027 16:14	vacances
	06/03/2026 00:58	28/01/2027 21:39	maladie

nom : Garcia prénom : Antoine

ajouter une absence

date de début : motif :

date de fin :

Ensuite je code le remplissage de la comboBox motif avec la liste des motifs présents dans la base de données, comme je l'ai fait avec le remplissage de la comboBox service de l'interface graphique FrmMediaTek86 (Gestion du Personnel).

J'ajoute ensuite le nom et prénom du personnel sélectionné dans l'interface graphique FrmAbsence car jusqu'à présent c'était un placeholder donc je crée la méthode RemplirNomPrenom() dans la classe FrmAbsence

```

/// <summary>
/// Permet de remplir la liste des prénoms et noms du personnel sélectionner pour pouvoir les afficher dans la fenêtre d'absence
/// </summary>
1 référence
private void RemplirNomPrenom()
{
    nomLabel.Text = personnelRecu.Nom;
    prnLabel.Text = personnelRecu.Prenom;
}

```

Je passe ensuite au codage de l'ajout d'une absence. Pour ce faire, ce sera le même principe que l'ajout d'un personnel, mais il faudra ajouter une vérification supplémentaire.

Il faut rajouter une vérification pour ne pas ajouter une absence qui empiéterait sur d'autres absences.

Pour cela, j'ai codé dans mon controller FrmAbsenceController une méthode qui sera appelé dans la view pour vérifier si une absence qui demande à être ajouté n'empiètera pas avec une autre.

```
-  
/// <summary>  
/// Permet de vérifier si un créneau d'absence est libre pour un personnel donné  
/// </summary>  
/// <param name="idPersonnel"></param>  
/// <param name="datedebut"></param>  
/// <param name="datefin"></param>  
/// <returns></returns>  
1 référence  
public bool CreneauLibre(int idPersonnel, DateTime datedebut, DateTime datefin)  
{  
    List<Absence> absences = GetAbsence();  
  
    foreach (Absence abs in absences)  
    {  
        if (datedebut <= abs.date_de_fin && datefin >= abs.date_de_debut)  
        {  
            return false;  
        }  
    }  
    return true;  
}
```

Si une absence a une date de début inférieur ou égal à la date de fin et si la date de fin a une date supérieur ou égal à une date de début qui est déjà présent dans la liste des absences, alors on retourne le fait que le créneau n'est pas libre.

Sachant que la logique derrière la création d'une absence est la même que celle derrière la création d'un personnel, je crée alors la méthode AddAbsence dans mon controller et ma dal qui permet donc de créer une absence sur un idpersonnel sélectionné en amont et un motif sélectionné dans la combobox, et je crée la méthode btnEnregAbs_Click dans la classe FrmAbsence en appelant la méthode CreneauLibre et en vérifiant que la date de fin est antérieure à la date de début.

```
private void btnEnregAbs_Click(object sender, EventArgs e)
{
    if (!dtpDebutAbs.Value.Equals("") && !dtpFinAbs.Text.Equals("") && cboMotif.SelectedIndex != -1)
    {
        if (dtpFinAbs.Value > dtpDebutAbs.Value)
        {
            if (controller.CreneauLibre(personnelRecu.Idpersonnel, dtpDebutAbs.Value, dtpFinAbs.Value))
            {
                Motif motif = (Motif)bdgMotifs.List[bdgMotifs.Position];
                Absence absence = new Absence(personnelRecu, dtpDebutAbs.Value, dtpFinAbs.Value, motif);
                controller.AddAbsence(absence);
                RemplirListeAbsence();
            }
            else
            {
                MessageBox.Show("Il existe déjà une absence pour ce créneau.", "Information");
            }
        }
        else
        {
            MessageBox.Show("La date de fin doit être postérieure à la date de début.", "Information");
        }
    }
    else
    {
        MessageBox.Show("Tous les champs doivent être remplis.", "Information");
    }
}
```

```
/// <summary>
/// Demande d'ajout d'une absence sur un personnel
/// </summary>
/// <param name="absence">objet absence à ajouter</param>
public void AddAbsence(Absence absence)
{
    if (access.Manager != null)
    {
        string req = "insert into absence(idpersonnel, datedebut, datefin, idmotif) ";
        req += "values (@idpersonnel, @datedebut, @datefin, @idmotif)";
        Dictionary<string, object> parameters = new Dictionary<string, object>();
        parameters.Add("@idpersonnel", absence.idpersonnel.Idpersonnel);
        parameters.Add("@datedebut", absence.date_de_debut);
        parameters.Add("@datefin", absence.date_de_fin);
        parameters.Add("@idmotif", absence.motif.Idmotif);
        try
        {
            access.Manager.RegUpdate(req, parameters);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }
}
```

```

/// <summary>
/// Demande d'ajout d'une absence
/// </summary>
/// <param name="absence">objet absence à ajouter</param>
1 référence
public void AddAbsence(Absence absence)
{
    absenceAccess.AddAbsence(absence);
}

```

Ensuite, même chose que dans la partie Personnel, j'ajoute le fait de pouvoir modifier une absence tout en respectant le fait que celle-ci n'empiète pas avec une autre avec la méthode CreneauLibre et en modifiant donc ma méthode btnEnregAbs_Click, tout en ajoutant les méthodes UpdateAbsence dans mon controller et ma dal. Je n'oublie pas aussi d'ajouter une méthode EnCoursModifAbsence en booléen, un booléen EnCoursDeModifAbsence et une méthode btnDemandeModifAbs_click qui permet de voir si une absence est sélectionné et ainsi de remplir les dates par la date de l'absence sélectionné et ainsi pouvoir la modifier si celle-ci n'empiète pas avec une autre. Il y a bien évidemment une confirmation de modification.

D'ailleurs, il faut penser à modifier la méthode CreneauLibre, puisque pour modifier une absence qui existe déjà, il est préférable de faire en sorte que l'absence sélectionné ne soit pas compté dans la liste des absences pour que l'on puisse la modifier d'un jour en plus vers la fin par-exemple. Et c'est possible grâce à l'ajout de la valeur datedebutavantmodif dans CreneauLibre qui s'appelle ancienneDate dans UpdateAbsence et dans FrmAbsence qui permet de stocker la date de début de l'absence sélectionné AVANT modification pour être sûr que ce soit la bonne absence à modifier ET qu'elle peut être modifier même si elle est sélectionné.


```

/// <summary>
/// Permet de vérifier si un créneau d'absence est libre pour un personnel donné
/// </summary>
/// <param name="idPersonnel"></param>
/// <param name="datedebut"></param>
/// <param name="datefin"></param>
/// <param name="datedebutavantmodif"></param>
/// <returns></returns>
1 référence | 0 modification | 0 auteur, 0 modification
public bool CreneauLibre(int idPersonnel, DateTime datedebut, DateTime datefin, DateTime? datedebutavantmodif)
{
    List<Absence> absences = GetAbsence();

    foreach (Absence abs in absences)
    {
        if (datedebutavantmodif.HasValue && abs.date_de_debut == datedebutavantmodif.Value)
        {
            continue;
        }

        if (datedebut <= abs.date_de_fin && datefin >= abs.date_de_debut)
        {
            return false;
        }
    }

    return true;
}

```

```

private void btnEnregAbs_Click(object sender, EventArgs e)
{
    if (!dtpDebutAbs.Value.Equals("") && !dtpFinAbs.Text.Equals("") && cboMotif.SelectedIndex != -1)
    {
        if (dtpFinAbs.Value > dtpDebutAbs.Value)
        {
            DateTime? ancienneDate = null;

            if (enCoursDeModifAbsence)
            {
                Absence absenceEnCours = (Absence)bdgAbsence.List[bdgAbsence.Position];
                ancienneDate = absenceEnCours.date_de_debut;
            }
            if (controller.CreneauLibre(dtpDebutAbs.Value, dtpFinAbs.Value, ancienneDate))
            {
                Motif motif = (Motif)bdgMotifs.List[bdgMotifs.Position];

                if (enCoursDeModifAbsence)
                {
                    Absence absence = (Absence)bdgAbsence.List[bdgAbsence.Position];

                    if (MessageBox.Show("Voulez-vous vraiment modifier l'absence du " + ancienneDate + " jusqu'au " + absence.date_de_fin + " de " + personnelRecu,
                    {
                        absence.idpersonnel = personnelRecu;
                        absence.date_de_debut = dtpDebutAbs.Value;
                        absence.date_de_fin = dtpFinAbs.Value;
                        absence.motif = motif;

                        controller.UpdateAbsence(absence, ancienneDate);
                    }
                }
                else
                {
                    Absence absence = new Absence(personnelRecu, dtpDebutAbs.Value, dtpFinAbs.Value, motif);
                    controller.AddAbsence(absence);
                }

                RemplirListeAbsence();
                EnCoursModifAbsence(false);
            }
            else

```

Suite du code

```

            EnCoursModifAbsence(true);
        }
        else
        {
            MessageBox.Show("Il existe déjà une absence pour ce créneau.", "Information");
        }
    }
    else
    {
        MessageBox.Show("La date de fin doit être postérieure à la date de début.", "Information");
    }
}
else
{
    MessageBox.Show("Tous les champs doivent être remplis.", "Information");
}
}
}

```

```

/// <summary>
/// Permet de changer le boolean enCoursDeModifAbsence pour pouvoir différencier une modification d'une création d'une absence et de changer le titre du group box en fonction de l'action en cours
/// </summary>
/// <param name="modif"></param>
2 références
private void EnCoursModifAbsence(Boolean modif)
{
    enCoursDeModifAbsence = modif;
    if (modif)
    {
        grbAbsence.Text = "modifier une absence";
    }
    else
    {
        grbAbsence.Text = "ajouter une absence";
        dtpDebutAbs.Value = DateTime.Now;
        dtpFinAbs.Value = DateTime.Now;
        cboMotif.SelectedIndex = 0;
    }
}

```

```

1 référence
private void btnDemandeModifAbs_Click(object sender, EventArgs e)
{
    if (dgvAbsences.SelectedRows.Count > 0)
    {
        Absence absence = (Absence)bdgAbsence.List[bdgAbsence.Position];
        EnCoursModifAbsence(true);
        dtpDebutAbs.Value = absence.date_de_debut;
        dtpFinAbs.Value = absence.date_de_fin;
        cboMotif.SelectedIndex = cboMotif.FindStringExact(absence.motif.Libelle);
    }
    else
    {
        MessageBox.Show("Une ligne doit être sélectionnée.", "Information");
    }
}

```

```

/// <summary>
/// Permet de vérifier si une absence est en cours de modification
/// </summary>
private Boolean enCoursDeModifAbsence = false;
///
/// <summary>
/// Demande de modification d'une absence
/// </summary>
/// <param name="absence">objet absence à modifier</param>
/// <param name="date">date d'avant à garder</param>
1 référence | 0 modification | 0 auteur, 0 modification
public void UpdateAbsence(Absence absence, DateTime? date)
{
    absenceAccess.UpdateAbsence(absence, date);
}

```

```

/// <summary>
/// Demande de modification d'une absence sur un personnel
/// </summary>
/// <param name="absence">objet absence à modifier</param>
/// <param name="ancienneDate">objet absence à modifier en gardant l'ancienne date pour pouvoir correctement changer la date</param>
1 référence | 0 modification | 0 auteur, 0 modification
public void UpdateAbsence(Absence absence, DateTime? ancienneDate)
{
    if (access.Manager != null)
    {
        string req = "update absence set datedebut = @datedebut, datefin = @datefin, idmotif = @idmotif ";
        req += "where idpersonnel = @idpersonnel and datedebut = @datedebutavant;";

        Dictionary<string, object> parameters = new Dictionary<string, object>();
        parameters.Add("@idpersonnel", absence.idpersonnel.Idpersonnel);
        parameters.Add("@datedebut", absence.date_de_debut);
        parameters.Add("@datefin", absence.date_de_fin);
        parameters.Add("@idmotif", absence.motif.Idmotif);

        parameters.Add("@datedebutavant", ancienneDate);

        try
        {
            access.Manager.ReqUpdate(req, parameters);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }
}

```


Etape 4

Le programme est désormais fini et fonctionnel. Je le test donc et me rend compte qu'il y a un bug : S'il n'y a aucun personnel dans la base de données de base, alors on ne peut pas en ajouter car l'index de la liste négatif. J'ai donc tout simplement défini mon personnel seulement s'il est crée ou seulement s'il est modifié, et non pas avant de savoir si celui est modifié ou crée.

```
Personnel personnel = (Personnel)bdgPersonnel.List[bdgPersonnel.Position];  
if (MessageBox.Show("Voulez-vous vraiment modifier les informations de " + personnel.Nom + " " + personnel.P  
{  
    personnel.Nom = txtNom.Text;  
    personnel.Prenom = txtPrenom.Text;  
    personnel.Tel = txtTel.Text;  
    personnel.Mail = txtMail.Text;  
    personnel.Service = service;  
    controller.UpdatePersonnel(personnel);  
}
```

e

```
Personnel personnel = new Personnel(0, txtNom.Text, txtPrenom.Text, txtTel.Text, txtMail.Text, service);  
controller.AddPersonnel(personnel);
```

Je me rend aussi compte lors de la relecture de mon code, j'ai oublié d'enlever « int idPersonnel » dans ma méthode CreneauLibre puisque celui-ci m'est inutile. Je l'utilisais à la base en faisant

```
return absenceAccess.GetAbsence(this.idPersonnel);
```

Juste au dessus de `foreach (Absence abs in absences)`, mais j'ai préféré ensuite utiliser `List<Absence> absences= GetAbsence();` pour une meilleure accessibilité.

Ainsi il fallait aussi enlever `personnelRecu.Idpersonnel` dans `if (controller.CreneauLibre(personnelRecu.Idpersonnel, dtpDebutAbs.Value, dtpFinAbs.Value)).`

Etape 5

Lors de la relecture de mon code et l'utilisation de mon programme, je me suis rendu compte de quelque chose d'illogique. Pourquoi les absences d'un personnel ne sont-elles

pas supprimés pour éviter un afflux de données inutile et non utilisables dans la base de données ?

J'ai donc créé une méthode `DelLesAbsences` dans le contrôleur `FrmMediaTek86Controller` et ma `dal AbsenceAccess` pour pouvoir supprimer l'intégralité des absences d'un personnel. Je l'ai tout simplement appelé dans la méthode `btnDemandeSuppPers_Click` juste avant de supprimer le personnel.

```
/// <summary>
/// Permet de supprimer un personnel de la base de données après confirmation de l'utilisateur
/// Permet de supprimer l'intégralité des absences d'un personnel en même temps
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
1 référence | Neeeme, il y a 8 heures | 1 auteur, 2 modifications
private void btnDemandeSuppPers_Click(object sender, EventArgs e)
{
    if (dgvPersonnel.SelectedRows.Count > 0)
    {
        Personnel personnel = (Personnel)bdgPersonnel.List[bdgPersonnel.Position];
        if (MessageBox.Show("Voulez-vous vraiment supprimer " + personnel.Non + " " + personnel.Prenom + " ?", "Confirmation de suppression", MessageBoxButtons.YesNo) == DialogResult.Yes)
        {
            controller.DelLesAbsences(personnel);
            controller.DelPersonnel(personnel);
            RemplirListePersonnel();
        }
    }
    else
    {
        MessageBox.Show("Veuillez sélectionner une ligne.", "Information");
    }
}

1 référence | Neeeme, il y a 8 heures | 1 auteur, 1 modification
```

```
/// <summary>
/// Suppression de l'intégralité des absences d'un personnel lorsque celui ci est supprimé
/// </summary>
/// <param name="personnel">objet absence sur personnel à supprimer</param>
1 référence | Neeeme, il y a 8 heures | 1 auteur, 1 modification
public void DelLesAbsences(Personnel personnel)
{
    absenceAccess.DelLesAbsences(personnel);
}
```

```

/// <summary>
/// Suppression de l'intégralité des absences d'un personnel lorsque celui ci est supprimé
/// </summary>
/// <param name="personnel">objet absence sur personnel à supprimer</param>

public void DelLesAbsences(Personnel personnel)
{
    if (access.Manager != null)
    {
        string req = "delete from absence where idpersonnel = @idpersonnel";
        Dictionary<string, object> parameters = new Dictionary<string, object>();
        parameters.Add("@idpersonnel", personnel.Idpersonnel);
        try
        {
            access.Manager.ReqUpdate(req, parameters);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }
}

```

J'ai donc retesté mon programme plusieurs fois et relu mon code plusieurs et cette fois c'est bon, le programme est prêt à être publié.

Etape 6

Il est maintenant l'heure de créer l'installateur du projet.

Je commence donc à ajouter un nouvel projet dans la solution. Celui-ci est « Setup Wizard. »

Je l'exécute et je sélectionne bien « Sortie Principale from MediaTek86 » et je termine.

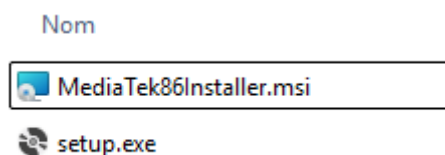
Je change l'auteur en mon nom et change le Manufacturer en MediaTek86. Je change aussi le ProductName en MediaTek86 pour que le logiciel ne se nomme pas « MediaTek86Installer »

Je sélectionne une icône que j'ajoute dans mon fichier MediaTek86Installer.

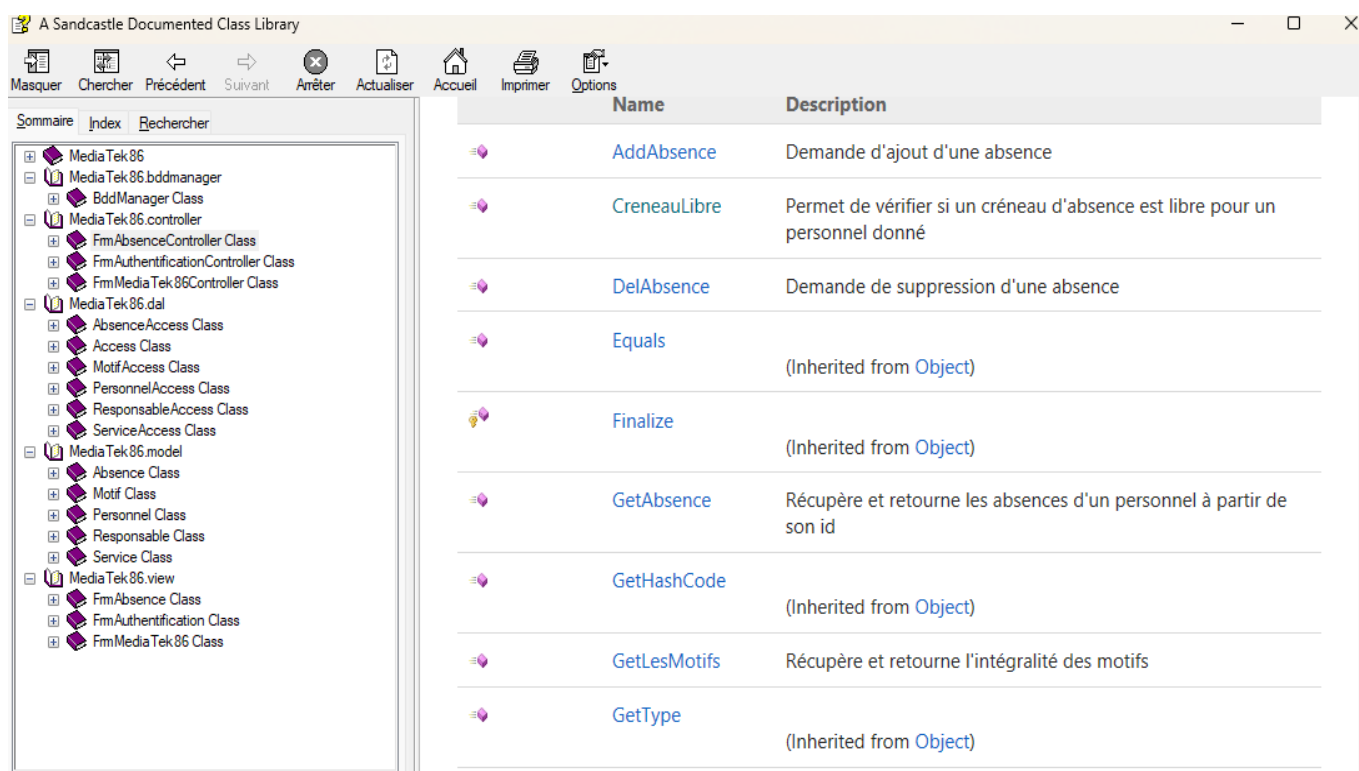
Je crée ensuite deux shortcuts que je renomme MediaTek86 et j'en met un de chaque dans User's Desktop et dans User's Programs Menu.

Je leur attribue aux deux l'icône que j'ai précédemment ajouté dans mon fichier.

Je pense à bien cocher les deux cases « Générer » dans le gestionnaire de configurations pour la configuration Release et Debug et je génère ensuite ma solution.



Je génère ensuite ma documentation finale à l'aide de SandCastle comme je l'ai fait précédemment au début du codage du programme.



Conclusion

Et voilà ! Le programme est officiellement fini et fonctionnel et peut maintenant être mis à la médiathèque pour le personnel.

La création de ce programme m'a permis une autonomie complète sur la création d'un programme de A à Z et c'était plutôt enrichissant comme atelier.

Cela m'a donc appris à mieux comprendre les étapes de développement d'une application et me permet de me rapprocher un peu plus du milieu professionnel.

J'ai pu renforcer donc mes capacités du langage C#, de l'utilisation de WinForms, de créer une architecture MVC moi-même, la gestion de base de données, utilisation de GitHub pour pouvoir sauvegarder l'intégralité du projet et voir toutes mes versions et créer une documentation technique au fur et à mesure du projet.